Les Tableaux Unidimensionnels et le Passage par Adresse en Langage C

Thème principal : Tableaux unidimensionnels, allocation mémoire contiguë, arithmétique des pointeurs et mécanisme d'Array Decay en langage C

Synthèse exécutive : Ce cours aborde la structure et la manipulation des tableaux à une dimension en langage C. Il détaille l'organisation de la mémoire vive (stack) sous forme de plages d'adresses contiguës, garantissant un accès direct à chaque élément en complexité temporelle constante O(1). Il explicite le mécanisme fondamental d'Array Decay (dégradation automatique du nom d'un tableau en un pointeur vers son premier élément lors du passage à une fonction), ainsi que les conséquences sur la perte de la dimensionnalité du tableau. Enfin, il formalise les règles de transmission par adresse aux fonctions et la nécessité absolue d'envoyer la taille du tableau en paramètre explicite.

1. Représentation mémoire, contiguïté et calcul d'adresse en O(1)

Lorsqu'un tableau statique est déclaré en C (par exemple `int tab[5];`), le compilateur réserve un bloc continu en mémoire vive sur la pile. Pour un tableau de 5 entiers codés sur 4 octets, 20 octets consécutifs sont alloués. L'accès à un élément via la syntaxe crochet `tab[i]` ne réalise aucun parcours séquentiel des cases précédentes : le processeur calcule directement l'emplacement exact en mémoire en additionnant à l'adresse de base du tableau le produit de l'indice par la taille en octets du type sous-jacent. Puisque l'élément initial ne présente aucun décalage par rapport au début du bloc alloué, son indice est obligatoirement 0 (`Adresse_Base + 0`). Il en résulte qu'un tableau de N éléments s'indexe exclusivement de 0 à N-1. Tenter d'accéder à l'indice N constitue un débordement de mémoire (Buffer Overflow) car le langage C n'effectue aucun contrôle des limites à l'exécution.

- Tableau unidimensionnel : Séquence homogène d'éléments de même type stockés sous une taille fixe prédéterminée.
- Contiguïté mémoire : Allotissement sur la pile d'un bloc ininterrompu de N * sizeof(type) octets sans aucun espace intermédiaire.
- Indexation de 0 à N-1 : L'indice i représente le décalage (offset) par rapport à l'adresse mémoire de départ.
- Complexité O(1) : Accès direct instantané grâce au calcul d'adresse système Adresse(tab[i]) = Adresse_Base + (i * sizeof(type)).
- Buffer Overflow : Risque d'accès hors limites (ex: tab[N] pour un tableau de taille N) causant un comportement indéterminé ou une Segmentation Fault.

Cas concret enseignant : *Le professeur a utilisé la métaphore d'un meuble de rangement vertical composé de tiroirs rigoureusement identiques et collés les uns au-dessus des autres, numérotés de 0 à N-1. Si l'on connaît le sol (l'adresse de base du meuble) et la hauteur exacte d'un tiroir (sizeof(type)), on calcule instantanément la hauteur exacte du tiroir i sans avoir besoin d'ouvrir ou de mesurer tous les tiroirs précédents.*

2. Passage de tableaux aux fonctions et mécanisme d'Array Decay

En langage C, le passage par valeur (copie intégrale de la structure) est désactivé pour les tableaux afin d'optimiser l'utilisation de la mémoire et la vitesse d'exécution. Dès qu'un tableau est passé en paramètre à une fonction, son nom subit une dégradation appelée 'Array Decay' : il se transforme en un simple pointeur `type*` pointant vers l'adresse du premier élément (`&tab;[0]`). Ce mécanisme a deux conséquences fondamentales : 1. Toute modification réalisée sur le tableau à l'intérieur de la fonction modifie directement le tableau d'origine situé dans la fonction appelante (passage par adresse). 2. Dans la fonction réceptrice, la notion de tableau disparaît au profit d'un pointeur. Par conséquent, l'opérateur `sizeof(tab)` dans la fonction renvoie la taille du pointeur (4 ou 8 octets selon l'architecture) et non la taille globale du tableau. Il est donc strictement obligatoire d'inclure la taille effective du tableau en tant que paramètre entier séparé dans la signature de la fonction.

• Array Decay : Conversion implicite et automatique du nom d'un tableau en un pointeur vers son premier élément (&tab;[0]) lors de son passage en argument.

• Passage par adresse : Transmission de l'emplacement mémoire original évitant la copie lourde de structures de données volumineuses.

• Arithmétique des pointeurs : Équivalence stricte entre l'écriture crochet tab[i] et la déréférenciation *(tab + i).

• Perte d'information de taille : Impossibilité pour une fonction de calculer dynamiquement la taille d'un tableau reçu via sizeof.

Cas concret enseignant : *Pour expliquer la différence entre passage par valeur et passage par adresse via l'Array Decay, le professeur a expliqué que transmettre un tableau à une fonction ne revient pas à fabriquer une réplique exacte de sa maison dans le jardin de son ami (passage par valeur), mais plutôt à lui donner les coordonnées GPS précises de sa propre maison (passage par adresse). L'ami utilise ces coordonnées pour se rendre sur place et repeindre directement les murs de la maison originale.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante-t-il ou affiche-t-il des valeurs absurdes lorsque j'essaie d'accéder à tab[5] alors que mon tableau a été déclaré avec int tab[5] ?
Réponse du professeur : Lorsqu'on déclare `int tab[5]`, la taille est de 5 éléments. Les indices valides vont strictement de 0 à 4. L'expression `tab[5]` tente d'accéder à la 6ème case mémoire, située en dehors de la plage réservée sur la pile. En C, il n'y a pas de vérification des bornes (bounds checking) : vous lisez ou écrivez dans une zone mémoire inconnue, provoquant soit une valeur garbage (absurde), soit une erreur de segmentation (Segmentation Fault).

Question étudiant : Est-ce qu'on peut calculer la taille d'un tableau à l'intérieur d'une fonction avec l'expression sizeof(tab) / sizeof(tab[0]) ?
Réponse du professeur : Non, c'est un piège classique en partiel ! Dans la fonction appelante (où le tableau est déclaré), `sizeof(tab) / sizeof(tab[0])` fonctionne. Mais une fois transmis à une fonction, le tableau subit l'Array Decay et devient un simple pointeur `int*`. Dans la fonction, `sizeof(tab)` renverra la taille du pointeur (ex: 8 octets sur système 64 bits), et `sizeof(tab[0])` la taille d'un int (4 octets). Le résultat donnera toujours 2, quelle que soit la vraie taille du tableau. C'est pourquoi la taille doit toujours être transmise en paramètre explicite.

■ Exercice & Démonstration d'Application

Énoncé : Écrire une fonction C nommée `remplir_et_calculer_moyenne` qui prend en paramètres un tableau d'entiers et sa taille effective. La fonction doit demander à l'utilisateur de saisir au clavier chaque élément du tableau, puis calculer et retourner la moyenne arithmétique exacte sous forme de nombre à virgule (`double`). La fonction doit être sécurisée en cas de taille invalide.

Résolution pas à pas :
1. Signature de la fonction : `double remplir_et_calculer_moyenne(int tab[], int taille)`. 2. Sécurisation : Vérifier si `taille <= 0`. Si vrai, retourner `0.0` pour éviter toute division par zéro. 3. Initialisation : Déclarer un accumulateur de type réel `double somme = 0.0;` pour éviter la troncature liée à la division entière. 4. Parcours et Saisie : Utiliser une boucle `for` avec un indice `i` allant de `0` à `taille - 1`. Utiliser `scanf("%d", &tab;[i]);` pour stocker la saisie et l'ajouter directement à `somme`. 5. Calcul et retour : Diviser `somme` par `taille` et renvoyer la valeur.

Résultat : #include double remplir_et_calculer_moyenne(int tab[], int taille) { if (taille <= 0) { return 0.0; } double somme = 0.0; for (int i = 0; i < taille; i++) { printf("Saisir la note [%d] : ", i + 1); scanf("%d", &tab;[i]); somme += tab[i]; } return somme / taille; } // Exemple d'exécution dans le main() : // int notes[3]; // double moy = remplir_et_calculer_moyenne(notes, 3); // Pour les saisies 12, 14, 16 -> La fonction retourne 14.000000

■ Auto-Évaluation Flash d'Examen

Q : Quel est l'indice du dernier élément accessible d'un tableau d'entiers déclaré par int t[10] ? Que se passe-t-il si l'on tente d'accéder à t[10] ?

R : L'indice du dernier élément est 9 (car l'indexation commence à 0). Tenter d'accéder à t[10] constitue un débordement de mémoire (Buffer Overflow) entraînant un comportement indéterminé ou un crash du programme.

Q : Que contient exactement l'identifiant d'un tableau lorsqu'il est employé seul sans crochets dans une expression C ?

R : Il contient l'adresse mémoire du premier élément du tableau (&tab;[0]), suite au mécanisme de dégradation automatique appelé Array Decay.

Q : Pourquoi la signature d'une fonction manipulant un tableau C doit-elle obligatoirement inclure sa taille en paramètre ?

R : Car lors du passage à une fonction, le tableau est converti en pointeur (Array Decay) et perd toute information sur sa taille d'origine. L'opérateur sizeof ne permet plus de la retrouver au sein de la fonction.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765